Exceptions

Exceptions: Concepts clés

- **Générer/lever une exception**
- **Prise en charge d'une exception**
- **Exceptions non prises en charge**
- **Objets exception: (information sur l'exception)**

Exceptions communes

IndexError: quand un entier n'est pas dans l'intervalle attendu:

```
>>> x = [2,3,4]
>>> x[3] provoque une IndexError
```

ValueError: quand un objet est d'un type adéquat mais dont la valeur n'est pas appropriée:

```
>>>  int("ballon") provoque une ValueError
```

KeyError: une recherche dans un dictionnaire avec une clé inexistante:

```
>>> codes = {'gb':44, 'us':1, 'no':47, 'fr': 33, 'es':34}
>>> codes['de'] provoque une KeyError
```

OSError: tout problème relié à un fichier (existence, lecture, écriture):

```
try:
    traiter_fichier("/tp/tp1/tp1.py")
except OSError as e:
    print(f"Erreur: {e}")
```

TypeError: quand le type n'est pas le bon:

```
>>> uneListe = [1,2,3]
>>> uneListe[1.2]  TypeError
```

KeyboardInterrupt: quand l'usager tape Control-C ou Delete

EOFError: essayer de lire quand on est en fin de fichier:

```
# test.py
x = input()
y = input()
$ echo "1 2" | python test.py
```

ZeroDivisionError: division par zéro

OverFlowError: quand le résultat d'une opération mathématique est trop grand

Exemple: convertir une liste en nombres

```python
#exceptionnel.py
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
      nombre = ''
      for s in nombres:
          nombre += DIGIT_MAP[s]
      x = int(nombre)
      return x


```

```
>>> from exceptionnel import convertir
>>> convertir ("un trois trois sept".split())

1337

>>> convertir ("un million sept".split())
  Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
    File "C:\Users\Michel\Downloads\scripts\exceptionnel.py", line 18, in convertir
      nombre += DIGIT_MAP[s]
  KeyError: 'million'

>>>
```

Exemple: convertir une liste en nombres (1)

```python
#exceptionnel1.py
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        x = int(nombre)
        print(f"Conversion réussie! x = {x}")
    except KeyError:
        print(f"Conversion manquée!")
        x = -1
    return x
```

```
>>> from exceptionnel1 import convertir
>>> convertir ("un trois trois sept".split())
  Conversion réussie! x = 1337
1337
>>> convertir ("un million sept".split())
  Conversion manquée!
-1
>>> convertir(512)
  Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
    File "C:\Users\Michel\Downloads\scripts\exceptionnel1.py", line 18, in convertir
      for s in nombres:
  TypeError: 'int' object is not iterable
>>>
```

Exemple: convertir une liste en nombres (2)

```python
#exceptionnel2.py
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        x = int(nombre)
        print(f"Conversion réussie! x = {x}")
    except KeyError:
        print("Mauvaise clé!")
        x = -1
    except TypeError:
        print("Mauvais type!")
        x = -1
    return x
```

```
>>> from exceptionnel2 import convertir
>>> convertir(512)
  Mauvais type!
-1

>>>
```

Exemple: convertir une liste en nombres (3)

```python
#exceptionnel3.py
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    x = -1
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        x = int(nombre)
        print(f"Conversion réussie! x = {x}")
    except (KeyError, TypeError):
        print("Conversion manquée!")
    return x
```

```
>>> from exceptionnel3 import convertir
>>> convertir(512)
  Conversion manquée!
-1
>>> convertir ("un million sept".split())
  Conversion manquée!
-1
>>>
```

Exemple: convertir une liste en nombres (4)

```python
#exceptionnel4.py
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    x = -1
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        x = int(nombre)
    except (KeyError, TypeError, IndentationError):

    return x
```

```python
# exceptionnel4x.py
try :
    import exceptionnel4
except(IndentationError):
    print("Mauvaise indentation!")
```

Console

```
>>> %Run exceptionnel4x.py

  Mauvaise indentation!

>>>
```

C'est une exception résultant du programmeur,. On n'intercepte généralement pas ces exceptions (IndentationError, SyntaxError et NameError). Ici, il suffit d'ajouter ***pass*** à la ligne 23 pour éviter l'exception.

```
>>> from exceptionnel4 import convertir
  Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
    File "C:\Users\miche\Downloads\scriptsExceptionnels\scripts\exceptionnel4.py", line 24
      return x
      ^^^^^^
  IndentationError: expected an indented block after 'except' statement on line 22
```

Exemple: convertir une liste en nombres (5)

```python
#exceptionnel5.py
import sys
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        return(int(nombre))
    except (KeyError, TypeError) as e:
        print(f"Conversion manquée: {repr(e)}", file = sys.stderr)
        #repr(objet) retourne la version imprimable de cet objet
        #repr() est généralement redéfinie pour cet objet
        return -1
```

```
>>> from exceptionnel5 import convertir
>>> convertir("un million sept".split())
  Conversion manquée: KeyError('million')
-1
>>> convertir(512)
  Conversion manquée: TypeError("'int' object is not iterable")
-1
>>>
```

Exemple: convertir une liste en nombres (6)

```python
#exceptionnel6.py
import sys
from math import log
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        return(int(nombre))
    except (KeyError, TypeError) as e:
        print(f"Conversion manquée: {repr(e)}", file = sys.stderr)
        return -1

#converti une liste ["un deux trois"] en son logarithme naturel 4.812184355372417
def string_log(s):
    v = convertir(s)
    return log(v)
```

```
>>> from exceptionnel6 import *
>>> string_log("zillion".split())
Conversion manquée: KeyError('zillion')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\Michel\Downloads\scripts\exceptionnel6.py", line 30, in string_log
    return log(v)
ValueError: math domain error
>>> log(-1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: math domain error
>>>
```

Exemple: convertir une liste en nombres (7)

```python
#exceptionnel7.py
import sys
from math import log
DIGIT_MAP = {
      'zéro':  '0',
      'un':    '1',
      'deux':  '2',
      'trois': '3',
      'quatre':'4',
      'cinq':  '5',
      'six':   '6',
      'sept':  '7',
      'huit':  '8',
      'neuf':  '9',
}
#converti une liste ["un", "deux", "trois"] en nombre entier 123
def convertir(nombres):
    try:
        nombre = ''
        for s in nombres:
            nombre += DIGIT_MAP[s]
        return(int(nombre))
    except (KeyError, TypeError) as e:
        raise # relance l'exception

#converti une liste ["un deux trois"] en son logarithme naturel 4.812184355372417
def string_log(s):
    try:
        v = convertir(s)
    except:
        return -1
    return log(v)
```

```
>>> %Run exceptionnel7.py
>>> from exceptionnel7 import *
>>> string_log("un two deux".split())

-1

>>> string_log("un trois deux".split())

4.882801922586371

>>>
```

Autre exemple: racine carrée d'un nombre

```python
#heron.py
def sqrt(x):
    """Calcul de la racine carrée avec la méthode d'Héron d'Alexandrie.
    Argument:
        x: le nombre à traiter.
    Retourne:
        La racine carrée de x.
    """
    approximation = x
    i = 0
    while approximation * approximation != x and i < 20:
        approximation = (approximation + x / approximation) / 2.0
        i += 1
    return approximation
def main():
    print(sqrt(9))
    print(sqrt(2))
    print(sqrt(-1))
if __name__ == '__main__':
    main()
```

Console

```
>>> %Run heron.py

3.0
1.4142135623730951
Traceback (most recent call last):
  File "C:\Users\Michel\OneDrive - Cégep André-Laurendeau\2021-A\555\getting-started-python-cor
    main()
  File "C:\Users\Michel\OneDrive - Cégep André-Laurendeau\2021-A\555\getting-started-python-cor
    print(sqrt(-1))
  File "C:\Users\Michel\OneDrive - Cégep André-Laurendeau\2021-A\555\getting-started-python-cor
    approximation = (approximation + x / approximation) / 2.0
ZeroDivisionError: float division by zero
```

Autre exemple: racine carrée d'un nombre (1)

```python
#heron1.py
import sys
def sqrt(x):
    """Calcul de la racine carrée avec la méthode d'Héron d'Alexandrie.
    Argument:
        x: le nombre à traiter.
    Retourne:
        La racine carrée de x.
     Exceptions levées:
        ValueError: Si x est négatif.
    """
    if x < 0:
        raise ValueError(f"Impossible de calculer la racine carrée de {x}")
    approximation = x
    i = 0
    while approximation * approximation != x and i < 20:
        approximation = (approximation + x / approximation) / 2.0
        i += 1
    return approximation
def main():
    try:
        print(sqrt(9))
        print(sqrt(2))
        print(sqrt(-1))
        print("Ligne impossible à atteindre")
    except ValueError as e:
        print(e, file=sys.stderr)
    print("L'exécution se poursuit ici.")
if __name__ == '__main__':
    main()
```

```
>>> %Run heron1.py

  3.0
  1.414213562373095
  Impossible de calculer la racine carrée de -1
  L'exécution se poursuit ici.

>>>
```

Gestion des exceptions avec *finally*

```python
#finalement.py
import os
import sys

def creer_dossier(chemin, dossier):
    chemin_original = os.getcwd()
    print(chemin_original)
    try:
        os.chdir(chemin)
        os.mkdir(dossier)
    except OSError as e:
        print(e)
    finally: # s'exécute indépendamment du résultat du bloc try
        os.chdir(chemin_original)
        print("retour au bercail de toute façon")
```

Console

```
>>> %Run finalement.py
>>> creer_dossier("chemin","fichier")

  C:\Users\Michel\Downloads\scripts
  [WinError 2] Le fichier spécifié est introuvable: 'chemin'
  retour au bercail de toute façon

>>> creer_dossier("chemin","fichier")

  C:\Users\Michel\Downloads\scripts
  retour au bercail de toute façon

>>>
```

en supposant ici qu'il n'y a pas d'exception parce qu'on a créé "chemin" entre-temps

Sommaire

- **Une exception est un objet Python qui représente une erreur.**
- **Lorsqu'une erreur est rencontrée dans un programme, l'interpréteur Python lève une exception.**
- **Les erreurs de syntaxe sont également traitées comme des exceptions, mais généralement non gérées/traitées.**
- **Lorsqu'une exception se produit pendant l'exécution d'un programme et qu'une exception intégrée est définie pour cela, le code écrit dans cette exception s'exécute.**
- **Le processus de gestion des exceptions implique l'écriture de code supplémentaire pour donner des messages ou des instructions appropriés à l'utilisateur. Cela empêche le programme de terminer brusquement.**
- **Lever une exception implique d'interrompre le flux normal de l'exécution du programme et de sauter au gestionnaire d'exceptions.**
- **Les instructions à l'intérieur du bloc *finally* sont toujours exécutées, qu'une exception se produise ou non dans le bloc *try*.**